RAG — Revision Notes

Retrieval-Augmented Generation

Modules 1–16

Simple → Example → Technical understanding → Interview revision

Purpose: A revision-friendly guide that follows the learning style used in the lessons. Future modules can be added in the same format.

1. Complete RAG Mental Model

```
OFFLINE / INDEXING
Documents
↓
Parsing / Cleaning
↓
Chunking
↓
Embedding
↓
Vector Store / Index

ONLINE / QUERY TIME
User Question
↓
Query Embedding
↓
Retrieval
↓
Reranking
↓
Context Construction
↓
Prompt
↓
LLM
↓
Answer + Citations
```

Core idea: RAG = SEARCH → GIVE FOUND INFORMATION TO THE LLM → GENERATE AN ANSWER.

RAG does not permanently teach or fine-tune the LLM. It retrieves external information at query time and places it into the model's context.

Module 1 — What is RAG?

Simple meaning: RAG stands for Retrieval-Augmented Generation. It combines information retrieval with an LLM so the model can answer using external, relevant information.

Example:

Question: “How many paid leaves does an employee get?” The system searches the employee handbook, retrieves the relevant section, and gives it to the LLM.

Technical understanding:

The retrieval component supplies evidence; the generation component produces natural-language output from the question plus retrieved context.

Interview points:

- RAG = Retrieval + Augmented + Generation.
- Retrieval finds relevant evidence.
- Generation produces the final response.
- RAG is not the same as fine-tuning.

Remember: RAG is like an open-book exam: search for the right pages, give them to the model, then let it answer.

Module 2 — RAG vs LLM

Simple meaning: An LLM generates text from its learned parameters and current prompt. RAG gives it additional external evidence at query time.

Technical understanding:

The LLM's parametric knowledge is learned during training. RAG provides non-parametric/external knowledge through retrieved context.

Interview points:

- LLM alone may not know private or newly updated company information.
- RAG can use private documents without retraining the model.
- RAG is especially useful when answers must be traceable to documents.

Remember: Without RAG: Question → LLM → Answer. With RAG: Question → Search → Context + Question → LLM → Answer.

Module 3 — Complete RAG Architecture

Simple meaning: RAG has an offline indexing phase and an online query phase.

Technical understanding:

Separating indexing from query-time retrieval avoids repeatedly processing the entire document collection for every question.

Interview points:

- Know the difference between indexing time and query time.
- Query-time work should be fast enough for interactive applications.
- Every stage can affect final answer quality.

Remember: Offline: documents are parsed, chunked, embedded and indexed. Online: the question is embedded, relevant chunks are retrieved and reranked, context is constructed, and the LLM answers.

Module 4 — Document Ingestion

Simple meaning: Document ingestion means bringing source documents into the RAG system.

Technical understanding:

Ingestion is the entry point into the indexing pipeline. Documents are collected and passed to parsing/cleaning before chunking.

Interview points:

- Ingestion is not the same as retrieval.
- The source type affects parsing strategy.
- Good ingestion preserves useful structure and metadata.

Remember: Sources may include PDFs, DOCX files, HTML pages, databases, knowledge bases, tickets, or other enterprise data.

Module 5 — Document Parsing

Simple meaning: Parsing means extracting usable content from documents.

Technical understanding:

Parsing may involve OCR for scanned documents, table extraction, layout analysis and metadata extraction.

Interview points:

- Bad parsing can corrupt the information before retrieval even begins.
- Scanned PDFs may require OCR.
- Preserve page/section metadata for later citations.

Remember: A PDF may contain text, headings, tables, page numbers and images. Parsing converts this into structured information the RAG pipeline can process.

Module 6 — Chunking

Simple meaning: Chunking means splitting a large document into smaller meaningful pieces.

Technical understanding:

Chunks should be large enough to preserve meaning but small enough to retrieve precisely and fit within context limits. Overlap can preserve continuity across boundaries.

Interview points:

- Too-small chunks can lose context.
- Too-large chunks can reduce retrieval precision and waste tokens.
- Common approaches: fixed-size, sentence, paragraph, recursive, semantic and structure-aware chunking.

Remember: Instead of embedding a 100-page handbook as one item, split it into sections such as leave policy, attendance, salary and benefits.

Module 7 — Chunk Size and Overlap

Simple meaning: Chunk size controls how much information is stored in each retrieval unit. Overlap repeats some text between neighboring chunks.

Technical understanding:

There is a tradeoff: larger chunks provide more context but can add noise; smaller chunks improve precision but may lose dependencies. Overlap helps boundary continuity but increases index size.

Interview points:

- There is no universal perfect chunk size.
- Tune chunking using the actual document type and evaluation results.
- Semantic/structure-aware boundaries are often better than arbitrary cuts.

Remember: If a sentence is split between two chunks, overlap can ensure both chunks contain enough surrounding context.

Module 8 — Embeddings

Simple meaning: An embedding is a numerical representation of text in a vector space.

Technical understanding:

The embedding model maps text to a vector such as [0.12, -0.31, ...]. Both document chunks and the user query must be represented in the same compatible embedding space for similarity search.

Interview points:

- Embedding model ≠ LLM generation model.
- Embeddings represent semantic information numerically.
- Document chunks are embedded during indexing; the query is embedded at query time.

Remember: “How do I reset my password?” and “I forgot my password; how can I change it?” can have vectors that are relatively close because their meanings are similar.

Module 9 — Vector Similarity

Simple meaning: Vector similarity measures how close two embeddings are in meaning space.

Technical understanding:

Cosine similarity: cos(A,B) = (A·B) / (||A|| ||B||). Euclidean distance measures geometric distance; lower distance generally means closer.

Interview points:

- Cosine similarity focuses on vector direction.
- Similarity/distance choice depends on the embedding model and index.
- The goal is to find nearest relevant vectors efficiently.

Remember: If two vectors point in similar directions, cosine similarity is high.

Module 10 — Vector Database / Vector Index

Simple meaning: A vector database stores embeddings together with their source text and metadata and supports efficient similarity search.

Technical understanding:

Exact nearest-neighbor search compares the query with every vector and can become expensive at scale. ANN (Approximate Nearest Neighbor) indexes such as HNSW and IVF reduce search cost.

Interview points:

- FAISS is a vector similarity-search library/indexing toolkit.
- Vector databases often add persistence, filtering, APIs and operational features.
- HNSW uses a navigable graph structure; IVF groups vectors into clusters.

Remember: A normal database can answer “employees where department = HR.” A vector search can find chunks semantically similar to “How many annual leaves are allowed?”

Module 11 — Retrieval

Simple meaning: Retrieval means finding information that may help answer the user's question.

Technical understanding:

Retrieval can be semantic/vector, keyword-based (such as BM25), hybrid, or constrained by metadata filters. Retrieval quality strongly affects downstream answer quality.

Interview points:

- Top-K is the number of candidates returned.
- Too-small K may miss needed evidence; too-large K adds noise and cost.
- Recall asks whether needed information was retrieved; precision asks how much of retrieved information is relevant.

Remember: Question → query embedding → vector search → top-K chunks.

Module 12 — Semantic Search

Simple meaning: Semantic search looks for meaning rather than requiring exact matching words.

Technical understanding:

The query and documents are embedded into the same vector space, then nearest-neighbor search identifies semantically similar items.

Interview points:

- Useful for paraphrases and natural-language questions.
- Exact identifiers, product codes and names can sometimes be better handled by keyword search.
- Semantic search and keyword search can be combined.

Remember: A query saying “How can I get my money back?” can retrieve a chunk titled “Refund procedure,” even if the word “money” does not appear.

Module 13 — BM25 and Hybrid Search

Simple meaning: BM25 is a keyword-based retrieval method. Hybrid search combines keyword matching with semantic/vector retrieval.

Technical understanding:

Hybrid systems may normalize scores and/or use rank fusion so candidates from different retrieval methods can be combined.

Interview points:

- Vector search is strong for meaning.
- BM25 is strong for exact terms.
- Hybrid retrieval can improve recall across mixed query types.

Remember: Query: “Model XJ-482 warranty.” Keyword search is strong for the exact identifier; vector search may help with related warranty language. Combining them can be stronger.

Module 14 — Reranking

Simple meaning: Reranking is a second-stage process that reorders the initial retrieved candidates using a more precise relevance model.

Technical understanding:

Embedding retrieval often uses independent query/document encodings (bi-encoder style). A cross-encoder reranker can jointly inspect query + candidate, usually improving relevance at higher compute cost.

Interview points:

- Retriever = broad and fast.
- Reranker = slower and more precise.
- A common mental model: retrieval aims for recall; reranking improves precision.

Remember: Retrieve 50 chunks quickly → reranker scores the 50 against the exact question → keep the best 5.

Module 15 — Context Construction

Simple meaning: Context construction means selecting, organizing, filtering and formatting retrieved/reranked information before sending it to the LLM.

Technical understanding:

It may include deduplication, relevance filtering, ordering, metadata preservation, context compression and token-budget management.

Interview points:

- Retrieved chunks are not automatically the final prompt context.
- Good context reduces noise, cost and latency.
- Metadata such as source/page/section enables traceable citations.
- Compression must preserve exceptions and conditions.

Remember: From 20 candidates, remove duplicates, discard irrelevant chunks, preserve useful metadata, order the evidence, and build a clean context.

Module 16 — Prompt Engineering for RAG

Simple meaning: Prompt engineering tells the LLM how to use the retrieved evidence and how to behave when evidence is missing.

Technical understanding:

A RAG prompt commonly contains instructions + retrieved context + user question. Grounding instructions encourage evidence-based answers. Abstention instructions reduce unsupported answers.

Interview points:

- Grounding = making the answer depend on provided evidence.
- Abstention = refusing to guess when evidence is insufficient.
- RAG reduces hallucinations but does not guarantee zero hallucinations.
- Prompt engineering cannot recover information that retrieval failed to retrieve.
- Treat retrieved documents as untrusted data; document text should not automatically be treated as trusted instructions.
- Citations should be tied to source metadata rather than blindly trusting generated citation text.

Remember: Instruction: “Answer using only the provided context. If the answer is not present, say the information is unavailable.” Context: “Employees receive 18 paid leaves.” Question: “How many leaves?”

Master Revision Sheet — One Page Mental Model

```
DOCUMENTS
↓
INGEST / PARSE
↓
CHUNK
↓
EMBED
↓
VECTOR STORE / INDEX
↓
USER QUESTION
↓
QUERY EMBEDDING
↓
RETRIEVE (semantic / BM25 / hybrid)
↓
RERANK
↓
CONTEXT CONSTRUCTION
↓
PROMPT
↓
LLM
↓
GROUNDED ANSWER
↓
CITATIONS / VALIDATION
```

Five words to remember:

- Chunking → break documents into useful pieces.
- Embedding → convert meaning into vectors.
- Retrieval → find candidate evidence.
- Reranking → choose the best evidence.
- Context + Prompt → prepare evidence and tell the LLM how to use it.

Most important distinction: Retriever finds information. Reranker orders information. Context construction prepares information. Prompt gives instructions. LLM generates the answer.

Golden rule: Better RAG is not just a better LLM. It is better data + parsing + chunking + retrieval + reranking + context + prompting + evaluation.

Future Modules — Continue in the Same Style

- Module 17 — RAG Generation
- Module 18 — Hallucinations in RAG
- Module 19 — Advanced RAG
- Module 20 — Parent-Child Retrieval
- Module 21 — Metadata Filtering
- Module 22 — Multimodal RAG
- Module 23 — Graph RAG
- Module 24 — Agentic RAG
- Module 25 — RAG Evaluation
- Module 26 — RAG Debugging

- Module 27 — Production RAG

Revision method: First read the Simple Meaning. Then redraw the pipeline from memory. Then explain the Example aloud. Finally answer the interview questions without looking at the notes.